# Backlog Sprints 5 & 6 — VIS Consulting

## Sprint 5 — Facturation complète : CRA → Facture → Paiement

1. Notification SSE « Facture en attente de génération » déclenchée après validation d'un CRA (événement `cra.validated`)
2. Génération facture en 1 clic : bouton dans dashboard admin crée PDF + Excel, statut initial `BROUILLON`, données vendeur pré-remplies depuis profil
3. Endpoint `PATCH /invoices/{id}/mark-sent` : passage statut `ENVOYÉE`, date d'envoi horodatée automatiquement
4. Endpoint `PATCH /invoices/{id}/mark-paid` : passage statut `PAYÉE`, saisie montant reçu + date paiement, calcul écart facturé vs reçu
5. Job quotidien `InvoiceScheduler` : détecte factures `ENVOYÉE` non payées à J+30, notification relance envoyée admin
6. Dashboard admin : liste factures par statut (`BROUILLON` / `ENVOYÉE` / `PAYÉE` / `ANNULÉE`), indicateur CA mensuel par tenant
7. Export CSV comptabilité : factures par période, format exploitable comptable externe
8. Vue platform_admin : chiffre d'affaires total, consultable par tenant et par mois
9. Règles de calcul : `invoiceDate` = 1er du mois suivant CRA ; `paymentDueDate` = `invoiceDate` + 45 jours, report au jour ouvré si week-end
10. Correctif : l'onglet Factures doit afficher les 2 factures générées si consultant a 2 projets sur le mois

## Sprint 6 — Observabilité & SLA : Prometheus, Grafana, Alertes

1. Déploiement Prometheus + Micrometer (namespace `monitoring`) : scrape `/actuator/prometheus` sur les 5 microservices, retention 30 jours
2. Dashboard Grafana « Platform Overview » : uptime, req/s, erreurs 5xx, CPU/RAM par pod, taux succès LLM
3. Dashboard Grafana « LLM Intelligence » : latence par modèle (Groq vs OpenAI), taux bascule fallback, taux erreur JSON
4. Dashboard Grafana « Business KPIs » : notes de frais (créées/validées), CRA (soumis/validés), factures (créées/envoyées/payées)
5. Dashboard Grafana « Kafka Health » : lag consumer, throughput, distribution partitions, taux rétention
6. Logs structurés JSON (Logback) + trace_id cross-service, centralisés dans Loki avec Promtail
7. Alertmanager : règles d'alerte (redémarrage pod > 3/h → CRITICAL, latence p95 > 1s → WARNING, lag Kafka > 100 → WARNING)
8. Alertmanager : routage vers Slack webhook + email, groupage par pod/service
9. Test de bout en bout : simuler crash pod, vérifier alerte Slack reçue en moins de 5 minutes
10. Runbook incident OPS dans `docs/RUNBOOK-OPS.md` : pod down, connexions DB saturées, circuit breaker ouvert, latence LLM excessive, lag Kafka
11. SLO à mesurer et exposer : disponibilité ≥ 99,5%/mois, latence p95 API ≤ 300ms, latence p95 Groq ≤ 3s / OpenAI ≤ 8s, délai SSE ≤ 2s, taux erreur 5xx ≤ 0,1%, succès extraction LLM ≥ 95%